

Restaurant Management System

Admin Dashboard — Complete Student Task List

This document contains the complete Backend, Security, Angular, Admin Dashboard, Search, Pagination, Redis, Internationalization, and Testing tasks.

1. Project Structure & Clean Code

- Create the project using a clean and maintainable architecture.
- Separate responsibilities into Controller, Service, Service Implementation, Repository, DTO, Mapper, Entity, Exception, and Configuration layers.
- Follow Clean Code principles throughout the application.
- Avoid duplicated business logic.
- Use meaningful names for classes, methods, variables, and API endpoints.
- Keep controllers responsible only for handling HTTP requests and responses.
- Keep business logic inside the service layer.
- Keep database operations inside the repository layer.
- Use DTOs for API requests and responses.
- Use Mapper classes to convert between Entity and DTO.
- Apply validation where required.
- Handle exceptions using a centralized exception-handling mechanism.

2. Category Management

- Create Category entity with ID, Name, Logo, and Flag.
- Configure Category → Product as One-to-Many.
- Configure Product → Category as Many-to-One.
- Create API to get all Categories.
- Create API to get Category by ID.
- Create API to save one Category.
- Create API to save a list of Categories.
- Create API to update one Category.
- Create API to update a list of Categories.
- Create API to delete Category by ID using RequestParam.
- Create API to delete multiple Categories using a list of IDs through RequestParam.
- Create Category Request DTO and Response DTO.
- Create Category Mapper.
- Apply Category validation.
- Return validation errors using bundled messages.
- Return Categories ordered by name ASC.

- Ensure list APIs are ordered by ID ASC where applicable.
- Implement Category search by name/key.
- Support searching by Category ID and key.
- Apply pagination to Category search APIs.

3. Product Management

- Create Product entity with ID, Name, Image Path, Description, and Price.
- Configure Category → Product as One-to-Many.
- Configure Product → Category as Many-to-One.
- Create API to get all Products.
- Create API to get Products by Category ID.
- Create API to get Product by ID.
- Create API to save one Product.
- Create API to save a list of Products.
- Create API to update one Product.
- Create API to update a list of Products.
- Create API to delete Product by ID using RequestParam.
- Create API to delete multiple Products using a list of IDs through RequestParam.
- Create Product Request DTO and Response DTO.
- Create Product Mapper.
- Validate Product name, description, price, and Category ID.
- Create Product search using a key for name or description.
- Support Category ID + search key.
- Apply pagination to Product search APIs.
- If no Products are found, return bundled English and Arabic messages.

4. Chef Management

- Create Chef entity with ID, Name, Specialization, Logo Path, Facebook Link, Twitter Link, and Instagram Link.
- Create API to get all Chefs.
- Create API to get Chef by ID.
- Create API to save Chef.
- Create API to update Chef.
- Create API to delete Chef.
- Apply pagination to Chef list APIs.
- Create Chef Controller, Service, Service Implementation, and Repository.
- Create Chef Request DTO and Response DTO.
- Create Chef Mapper.
- Apply validation and bundled messages.

5. Contact Information Management

- Create ContactInfo entity with ID, Name, Email, Subject, and Message.
- Create API to save Contact Information.
- Create API to get Contact Information.
- Create API to get Contact Information by ID.
- Create API to delete Contact Information.
- Apply pagination when returning multiple records.
- Create Controller, Service, Service Implementation, Repository, DTOs, and Mapper.
- Apply Controller-level validation.
- Validate Name, Email format, Subject, and Message.
- Return validation errors using English and Arabic bundled messages.

6. User, UserDetails & Role Management

- Create User entity with ID, Username, and Password.
- Create UserDetails entity with ID, Age, Phone Number, Address, and other required information.
- Create Role entity with ID and Code.
- Create Role enum with ADMIN and USER.
- Configure User → UserDetails as One-to-One.
- Configure UserDetails → User as One-to-One.
- Configure User → Role as Many-to-Many.
- Configure Role → User as Many-to-Many.
- Ensure role mapping supports authentication and authorization.

7. User & ContactInfo Refactoring

- Refactor User → ContactInfo as One-to-Many.
- Refactor ContactInfo → User as Many-to-One.
- Update Entity relationships.
- Update DTOs.
- Update Mappers.
- Update Services and Controllers.
- Verify existing APIs after refactoring.

8. Bundled Messages & Exception Handling

- Create centralized bundled-message support.
- Support English and Arabic messages.
- Create messages for successful operations.
- Create messages for validation failures.
- Create messages for missing data.

- Create messages for invalid IDs.
- Create messages for duplicate data.
- Create messages for authentication and authorization failures.
- Apply bundled messages consistently across all APIs.
- Create centralized exception handling.
- Handle Resource Not Found, Validation, Invalid Request, Duplicate Data, Authentication, Authorization, Pagination, and Empty Search Results.

9. Pagination & Sorting

- Apply pagination to all APIs that return lists.
- Start page numbering from 1.
- Define and validate page size.
- Prevent page number from being less than 1.
- Validate pagination inside the Service layer.
- Return bundled messages for invalid pagination.
- Return pagination metadata in list responses.
- Ensure paginated results have consistent ordering.
- Apply pagination to Categories, Products, Chefs, Contact Information, Orders, and Users where applicable.

10. Order Management

- Create Order entity with ID, Code, Total Price, and Total Number.
- Configure Order → Products as Many-to-Many.
- Configure Order → User as Many-to-One.
- Configure User → Orders as One-to-Many.
- Create API to create an Order using a list of Product IDs, Total Price, and Total Number.
- Get the logged-in User from the security context.
- Associate the Order with the authenticated User.
- Validate Product IDs, Total Price, and Total Number.
- Prevent creating Orders with invalid Products.
- Create API to get Orders for the logged-in User.
- Create API to get a specific Order for the logged-in User.
- Prevent users from accessing another user's Orders.
- Create Admin API to get all Orders.
- Create Admin API to get Order by ID.
- Apply pagination and ID ordering.

11. Audit Fields

- Add createdAt.
- Add updatedAt.

- Add createdBy.
- Add updatedBy.
- Configure automatic creation and update timestamps.
- Configure automatic created-by and updated-by handling.
- Apply the required audit annotations/configuration.
- Prevent normal API users from manually modifying audit fields.

12. Order Search

- Search Orders by creation-date range.
- Search Orders by update-date range.
- Search Orders by createdBy.
- Search Orders by updatedBy.
- Search Orders by Creation Date + Created By.
- Search Orders by Update Date + Updated By.
- Support multiple search criteria in one request.
- Validate all search criteria.
- Apply pagination.
- Return bundled messages when no Orders are found.

13. Product & Category Audit Search

- Search Products by creation-date range.
- Search Products by update-date range.
- Search Products by createdBy.
- Search Products by updatedBy.
- Search Products using multiple criteria.
- Search Categories by creation-date range.
- Search Categories by update-date range.
- Search Categories by createdBy.
- Search Categories by updatedBy.
- Search Categories using multiple criteria.
- Support date range + createdBy and date range + updatedBy.
- Apply pagination and bundled no-data messages.

14. Product Details

- Create ProductDetails entity/model with the required fields.
- Configure Product → ProductDetails as One-to-One.
- Configure ProductDetails → Product as One-to-One.
- Create Repository, Service, Service Implementation, and Controller.
- Create Request DTO and Response DTO.

- Create Mapper.
- Create API to add Product Details.
- Create API to get Product Details.
- Create API to update Product Details.
- Create API to delete Product Details.
- Validate Product ID.
- Prevent duplicate Product Details for the same Product.

15. Security — JWT Authentication

- Implement JWT-based authentication.
- Configure JWT generation and validation.
- Configure token expiration.
- Secure protected APIs.
- Keep Sign-Up and Login APIs public.
- Create public Sign-Up API.
- Create public Login API.
- Create User account and UserDetails during registration.
- Assign the default USER role.
- Encrypt passwords before storing them.
- Generate JWT after successful Login.
- Return authentication errors using bundled messages.

16. Authentication Validation Rules

- Username must not be empty.
- Username must contain at least 7 characters.
- Password must contain at least 7 characters.
- Password must contain at least one uppercase letter.
- Password must contain at least one lowercase letter.
- Password must contain at least one number.
- Password must contain at least one special character.
- Age is mandatory.
- Age must be between 16 and 80 inclusive.
- Return Arabic and English validation messages.

17. Authorization

- Configure ADMIN role.
- Configure USER role.
- Protect Admin APIs using ADMIN authorization.
- Protect User APIs using USER authorization where required.

- Prevent normal users from accessing Admin APIs.
- Return proper authorization error messages.

18. Angular Authentication & Routing

- Create Login component.
- Create Sign-Up component.
- Create Login and Sign-Up forms.
- Validate Login and Sign-Up forms.
- Connect Login to the Backend authentication API.
- Receive and handle JWT.
- Redirect to Product page after successful Login.
- Connect Sign-Up to the Backend registration API.
- Display success/failure messages.
- Redirect to Login after successful account creation.
- Create routes for Login, Sign-Up, Product, Product Details, Order, and Admin Dashboard.
- Protect authenticated routes.
- Protect Admin routes.
- Redirect unauthenticated users to Login.

19. Angular Product & Product Details

- Create Product search UI.
- Search Products by name.
- Search Products by description.
- Search Products by Category ID.
- Support Category ID + search key.
- Apply pagination.
- Display no-data messages.
- Create Product Details component.
- Add Product Details.
- Update Product Details.
- Delete Product Details.
- Display Product Details.
- Handle Front-End validation and Backend errors.

20. Internationalization

- Create English bundled messages.
- Create Arabic bundled messages.
- Support Arabic and English on the Front-End.
- Display Backend bundled messages correctly.

- Support switching between Arabic and English.
- Translate validation, success, error, authentication, authorization, and no-data messages.

21. Redis Integration

- Integrate Redis into the application.
- Configure Redis connection.
- Configure Redis serialization/deserialization.
- Apply caching where appropriate.
- Cache frequently requested data.
- Configure cache expiration.
- Invalidate or update cache after Create operations.
- Invalidate or update cache after Update operations.
- Invalidate cache after Delete operations.
- Verify that stale data is not returned.

22. Admin Dashboard

- Create Admin Dashboard.
- Display total Categories.
- Display total Products.
- Display total Chefs.
- Display total Users.
- Display total Orders.
- Display recent Orders.
- Display recent Products.
- Display recent Categories.
- Create Category Management page.
- Create Product Management page.
- Create Chef Management page.
- Create User Management page.
- Create Order Management page.
- Create Contact Information Management page.
- Create Product Details Management page.
- Add search, filtering, pagination, Create, Edit, and Delete actions.
- Display delete confirmation.
- Display success and error messages.

23. Postman Testing

- Test all Category APIs.
- Test all Product APIs.

- Test all Chef APIs.
- Test all ContactInfo APIs.
- Test User and Role APIs.
- Test ProductDetails APIs.
- Test Order APIs.
- Test Search APIs.
- Test Pagination and Sorting.
- Test Validation and Exception Handling.
- Test JWT generation and protected APIs.
- Test ADMIN and USER authorization.
- Test Arabic and English bundled messages.
- Test no-data and invalid-ID scenarios.
- Test Create, Update, Delete, and Bulk operations.
- Test multiple search criteria.
- Verify HTTP status codes and response structures.
- Verify database changes after each operation.

24. Final Integration & Quality Check

- Verify all Entity relationships.
- Verify all DTOs and Mappers.
- Verify Controllers, Services, and Repositories.
- Verify validation and exception handling.
- Verify bundled messages.
- Verify pagination and sorting.
- Verify searching and filtering.
- Verify JWT security and role-based authorization.
- Verify Redis.
- Verify audit fields.
- Verify Angular integration.
- Verify Arabic/English support.
- Verify Admin Dashboard.
- Verify all APIs through Postman.
- Remove duplicated and unused code.
- Apply consistent naming.
- Apply Clean Code principles.
- Verify all features work together end-to-end.